

Rashomon: A Desktop Environment Built on a Persistent Graph

Architecture Overview

Aseem Ratha

September 25, 2026

Contents

1	Core Model	2
2	Layered overview	4
3	Nodes: roles vs. types	4
3.1	Built-in types (ship with the kernel)	5
3.2	Example third-party types	6
4	Edges	6
5	Identity: entities and occurrences	7
6	Primitives	7
7	Self-hosting	8
8	Packaging and distribution	8
9	Concurrency	8
10	Tech stack	9
11	Worked example	9
12	Open / deferred decisions	9

A desktop environment built around a persistent graph of information, rather than conventional applications and files. Anything encountered while computing (a webpage, a note, a conversation, a terminal session, a piece of media) can become a persistent, addressable thing in the graph. Programs become swappable, composable ways of encountering and manipulating that graph, rather than owners of their own private state.

1 Core Model

Term	Meaning
Rashomon	the whole system
Kernel	the native process owning the graph, identity resolution, persistence, primitives, the component runtime, and the package manager
Node	the generic graph vertex. Every Node has a role : entity or occurrence
Entity	a Node with durable identity: a recurring page, a task, a topic, a browser profile
Occurrence	a Node representing one instance of running into something: this tab load, this shell session, this chat exchange. Weak/no identity of its own; always has exactly one occurrence-of edge to an Entity
Edge	a typed, directed link between two Nodes, carrying a timestamp and a confidence score
Facet	a declared way an Entity can be viewed or acted on (browser, notes, history, graph, terminal...)
View	one live, on-screen instance of a Facet being used right now. Facet = capability; View = an open instance of using it
Component	the sandboxed WASM binary implementing zero or more Facets' logic, and/or declaring, via its manifest, zero or more node/edge types it introduces (at least one of the two). Types are inert schema; only Facets carry behavior. Never implements a Primitive itself, only declares which ones it requires
Native Backend	the native, unsandboxed, kernel-bundled code that actually implements a given Primitive — e.g. CEF backing rashomon:browser , a PTY bridge backing rashomon:process . Part of the trusted kernel, not the installable Component ecosystem, despite sometimes shipping as a separate binary/process
Window	the OS-level surface a View is drawn in
Primitive	a host-implemented capability a Component can be granted access to (graph read/write, process spawn, browser control, network, storage...). Its backend is always a Native Backend, never an ordinary Component

One sentence: **the graph is made of Nodes (Entities and Occurrences) connected by Edges; an Entity exposes Facets, each implemented by a Component; opening a Facet creates a View, which is drawn in a Window; Components talk to the kernel only through Primitives.**

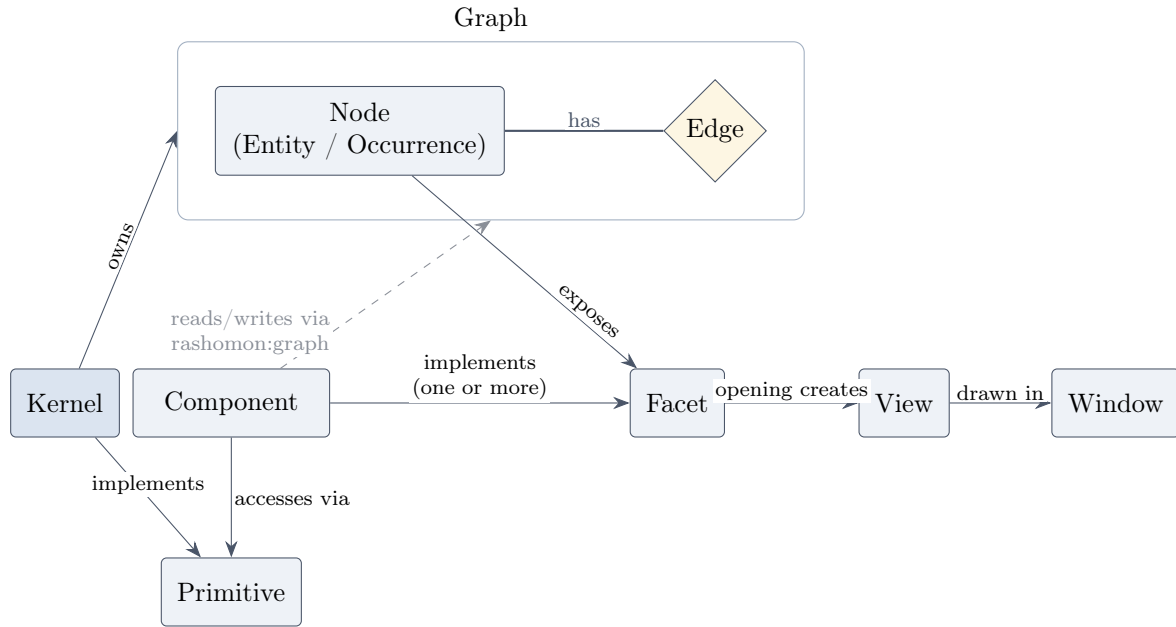


Figure 1: Core concepts and how they relate: the Kernel, Nodes, Edges, Facets, Views, Components, and Primitives.

2 Layered overview

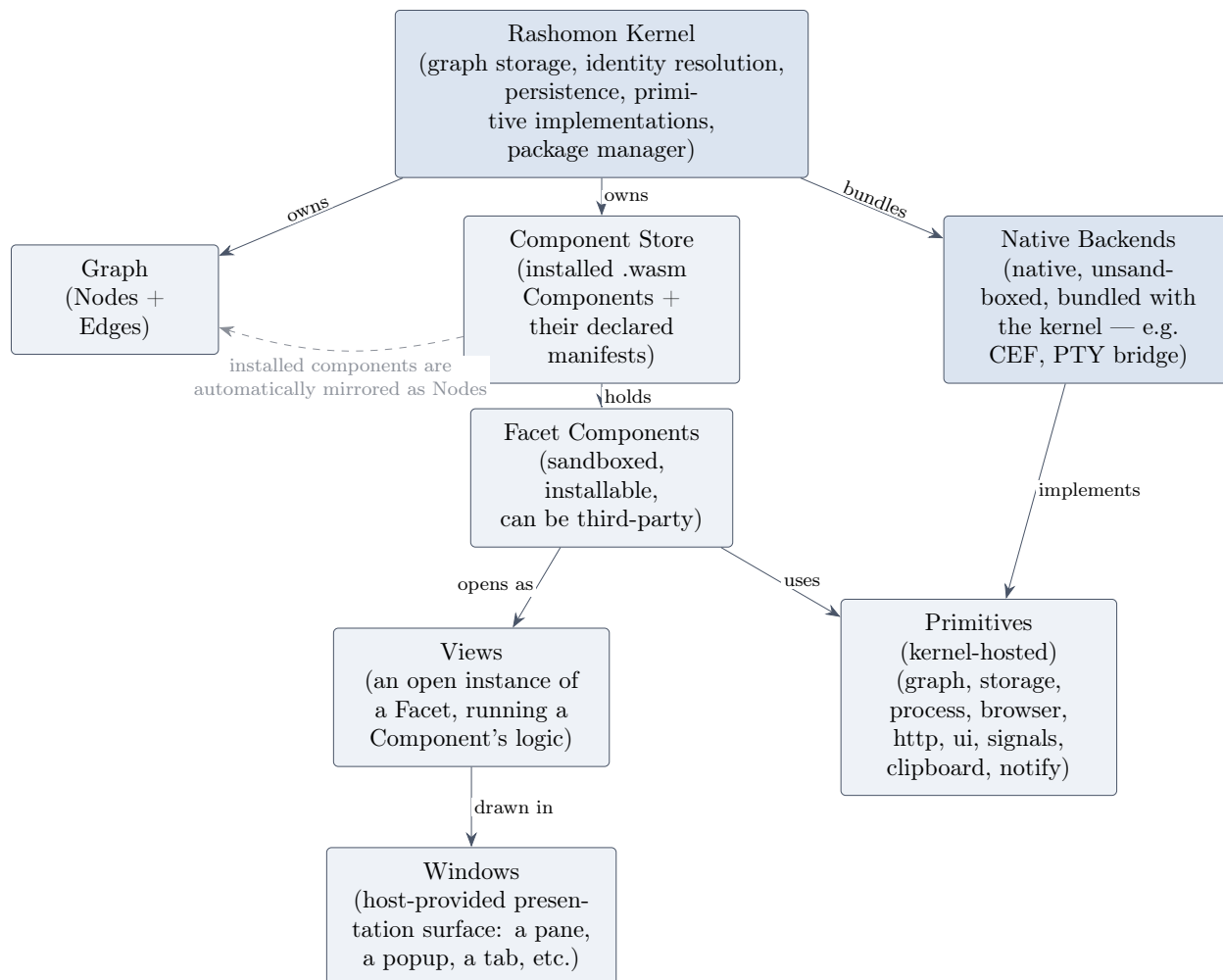


Figure 2: What the kernel directly owns (Graph, Component Store, Native Backends) versus what Facet Components provide on top (Views, Windows).

Hosts (an extension, an Electron/CEF shell, eventually a browser fork) sit outside this entirely. They only provide Windows and route user input in. Swapping a host should never touch the graph, the primitives, or any Component.

3 Nodes: roles vs. types

entity and **occurrence** are the only two **roles**, fixed by the kernel. This is what determines identity-resolution behavior and which structural edges apply. Concrete node **types** are open and component-defined, the same way Facets are.

A component's manifest registers the node types it introduces:

```
node-type shell-session {
```

```

    role: occurrence
    properties: { cwd: string, started-at: timestamp, exit-code: option<s32> }
}

```

Types are namespaced (`rashomon:page`, `someauthor:shell-session`) so two components can't collide on the same name.

A type is deliberately just a shape: a **role** plus a **properties** block, nothing else. It has no attached behavior, so the kernel (and any Facet, whether or not it's the one that introduced the type) can always safely store and inspect it. All actual logic — rendering, proposing identity-resolution keys, interpreting edges — lives in Facets, never in a type declaration.

Because of that split, implementing a Facet, introducing a node type, and introducing an edge type are three **independent** things a Component's manifest can declare, not a package deal. A Component can be as small as a single new Facet over an **existing** type (a "kanban board" Facet grouping ordinary `thread` Entities by status, introducing no new type at all), or just a new type with no dedicated Facet (relying on a generic outline/thread view to show it), or a full feature bundling a new type with every Facet it needs. Here's the fuller end of that spectrum — a hypothetical chat Component that both introduces a type and implements several Facets over it:

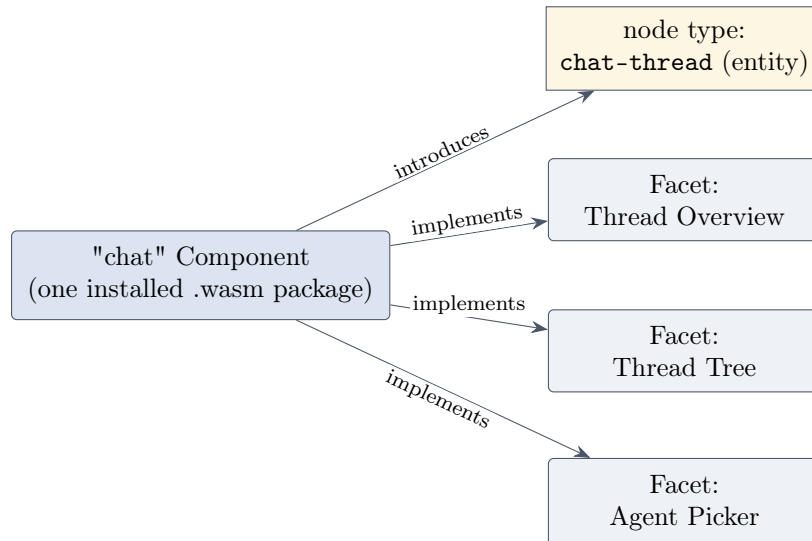


Figure 3: One possible shape of a Component's declared surface — a new node type plus every Facet it implements over it. A Component just as easily declares only one of these, or introduces no new type at all. Edge types (not pictured) work the same way.

3.1 Built-in types (ship with the kernel)

Type	Role	Notes
<code>thread</code>	entity	generic, no-schema; the default place to file anything with no more specific type
<code>page</code>	entity	a recurring web reference; identity resolved via URL + content similarity
<code>browser-context</code>	entity	a persistent browser profile/session identity

3.2 Example third-party types

Type	Role	Notes
shell-session	occurrence	one PTY-backed terminal run
dev-task	entity	optional richer entity a terminal component could define if generic thread isn't enough (tracks cwd/branch across sessions)
chat-thread	entity	optional richer entity a chat component could define if generic thread isn't enough (adds participant/topic keys for identity resolution)
chat-exchange	occurrence	one specific conversation instance under a chat-thread
installed-component	entity	the kernel's own package manager mirrors each installed Component as an ordinary Node, so it's queryable/browsable like anything else

4 Edges

Structural (kernel reasons about these directly):

- **Occurrence-of**: Occurrence $\rightarrow$ Entity. Mandatory, exactly one per Occurrence.
- **Contains**: Entity $\rightarrow$ Entity. The outline/tree hierarchy (threads containing pages, pages containing markers). Single-parent by default, a strict tree rather than a DAG, so outline-style Facets can render it simply. A second relationship uses **references** instead.
- **Supersedes / Diverged-from**: Entity $\rightarrow$ Entity. Produced by the identity-resolution ladder when a fuzzy match falls below the merge threshold, so a bad automatic call is correctable, not silently wrong.

Typed, open (kernel stores/queries generically; Components interpret):

- **References**: general "this points at that." An anchor is a specialization: **references** carrying an **anchor-spec** payload (quote + context, element descriptors, spatial position, or a media locator).
- **Context-of**: Occurrence $\rightarrow$ Entity, for "this happened under this browser-context/profile."
- **Derived-from**: provenance for generated content (a chat reply derived from the anchors in its context; a promoted note derived from a message).
- **Related-to**: a deliberately weak, manual "these matter to each other." The intended mechanism for surfacing an Entity somewhere beyond its one **contains** parent, without relaxing **contains**'s single-parent tree. Undirected in practice, either side can create it, and an outline-style Facet is expected to render an Entity's **related-to** links as a secondary "Related" section alongside its primary **contains** children, not merged into the tree itself.

Edge types beyond this built-in set are open the same way node types are: declared in a Component's manifest, namespaced the same way (**rashomon:references**, **someauthor:blocks**), so a code-review Component could introduce its own **blocks** or **reviewed-by** edge type between issue Entities without touching the kernel.

Every edge carries a **timestamp** and a **confidence score** (1.0 for anything manual; a real number from the similarity check for automatic **supersedes** edges).

5 Identity: entities and occurrences

Resolution is a per-occurrence matching problem, not a global definition of "what a webpage is." A **ladder**, run per occurrence against candidates a Facet's own matcher proposes:

1. **Strong key match**: a stable identifier (content hash, commit SHA, message ID). Exact match $\rightarrow$ same Entity, new version if content differs.
2. **Normalized-key match + similarity check**: e.g. a normalized URL. If an Entity already claims the key, diff new content against the last version. Above threshold $\rightarrow$ new version of the same Entity. Below threshold $\rightarrow$ new Entity, linked via **supersedes**~/~**diverged-from**.
3. **No match** $\rightarrow$ new Entity.

Keys are typed and Facet-defined, not hardcoded in the kernel: a browser Facet proposes URL-based keys, a terminal Facet proposes directory+branch keys, a chat Facet proposes participant+topic keys. Every merge/split is logged and reversible.

6 Primitives

A fixed, versioned WIT package every Component's **world** is composed from. Semver'd independently of the kernel's internals. This is the contract a future maintainer only needs to keep stable, not every Facet ever published.

- **rashomon:graph**: read/write Nodes and Edges; propose-match calls for the identity ladder.
- **rashomon:storage**: a sandboxed scope per Component.
- **rashomon:process**: spawn/manage OS processes via a **resource process** handle (stream in/out, resize, signal, wait). Its Native Backend owns the actual **exec()**; a Component never gets a raw syscall.
- **rashomon:browser: resource browser-context** (a persistent profile: cookies, cache, logins, itself represented as a **browser-context** Entity) and **resource browser-tab** (navigate, DOM snapshot, inject script, anchor, downloads) — a low-level handle for one browsing surface, not to be confused with a Core Model **View**. Backed by CEF as its Native Backend; never embedded in WASM itself (too heavy, needs native GPU/windowing).
- **wasi:http**: standard, reused rather than wrapped.
- **rashomon:ui**: a render-surface contract: a Component hands back a description of what to draw; the host draws it.
- **rashomon:signals**: subscribe to graph changes, timers, lifecycle.
- **rashomon:clipboard** / **rashomon:notify**: small, obviously-scoped.

This set is closed, not extensible the way Facets and node/edge types are: a Component can only ever **require** a Primitive (declare it as an import), never implement one. Every Primitive's actual backend is a Native Backend, native and unsandboxed by necessity, since a sandboxed WASM binary has no ambient authority to spawn processes or drive a browser engine on its own. Some

Native Backends are inline in the kernel binary itself; others (CEF, a PTY bridge) ship as separate native processes for isolation, but both are part of the trusted kernel distribution, not the installable Component ecosystem. Whether a narrower, genuinely lower-risk subset of Primitives (`rashomon:clipboard`, `rashomon:notify`) should ever open up to third-party-supplied backends is an open question, not the current design — it would mean auditing a very different threat model than "sandboxed Component with a declared world."

A Component's declared world (its imports) is static and inspectable before install: the permission-prompt story, shown the way an extension's requested permissions are shown today.

Decide `rashomon:process` and `rashomon:browser` against a real Facet that needs them, not in the abstract. Designing a capability before the first consumer exists is a common way these interfaces end up wrong.

7 Self-hosting

State lives in the **graph**, not inside a Component instance. A Component is closer to a stateless request handler over persistent Nodes than a live object with private state. That's what makes **hot-swap** (replace a running instance with a freshly compiled one) practical, even though true Lisp/Smalltalk-style **hot-edit** (mutating running code in place) isn't available to a compiled, sandboxed WASM binary.

The core design rule: **anything the kernel needs to show or manage about itself must first be representable as Nodes and Edges**, so managing it can be delegated to an ordinary Component. The node-type registry browser, the package manager, and the outline/thread view should all be installed Components from day one (not privileged native UI), or the system never actually becomes self-hosting later.

A REPL/scripting Facet is a plausible later addition (Nyxt's Lisp REPL, Emacs's `eval-buffer` are the precedent) but is **deferred**, not required while Components can already be written in any supported language, and worth designing only once the friction of not having one is concrete.

8 Packaging and distribution

A package = one `.wasm` file + a manifest declaring node types, Facets, edge types, and required primitives, the direct equivalent of a `.vsix`.

- **Local dev loop:** load straight from a directory (VS Code's "install from location" pattern), skipping the registry round-trip while iterating.
- **Distribution:** skip `warg~/~wa.dev` for now (still early) in favor of a signed-manifest index hosted on GitHub, browsed through an Extensions Facet (itself a Component) that shows requested capabilities before install: fetch, verify hash/signature, unpack, register. No config file editing, ever, for the install path itself.
- Revisit `warg` once it matures; the manifest format should be designed so migrating onto it later doesn't require re-authoring packages.

9 Concurrency

- **Across Views/Components:** each active View gets its own Wasmtime Store. I/O-bound work (watchers, streaming chat, browser events) runs as async tasks sharing an executor

(Wasmtime's component-model-async support). CPU-bound work dispatches its `Store`'s call onto a worker thread so it doesn't stall the executor.

- **Within a single Component:** true in-guest multithreading needs the WebAssembly "shared-everything-threads" proposal, which is still immature. **Don't build around it.** Instead, a Component that needs real parallel compute calls a **primitive** backed by a Native Backend (multithreaded Rust on the host side) and awaits the result. This is the same host-mediated pattern as `rashomon:process` and `rashomon:browser`.

10 Tech stack

- **Kernel:** Rust.
- **WASM runtime:** Wasmtime, the Bytecode Alliance reference implementation, furthest along on the Component Model and WASI Preview 2.
- **Browser primitive Native Backend:** CEF via `cef-rs` (the Tauri-maintained fork), actively tracked against current Chromium releases, ships its own binary-download and app-bundling tooling.
- **Component authoring:** any language with WIT bindings. Rust (`cargo-component`) is most mature; JS (`jco`), Python (`componentize-py`), and Go (`TinyGo`) are workable; C/C++ guest support is real but currently the least polished.
- **UI:** HTML-based, described by Components through `rashomon:ui` and drawn by the host, not native per-toolkit UI code.

11 Worked example

A thread called "researching CRISPR delivery methods":

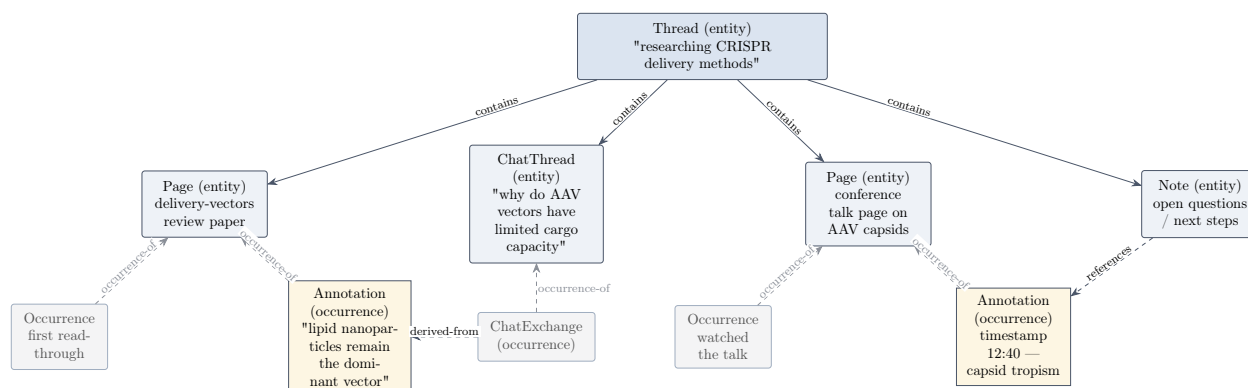


Figure 4: A thread ("researching CRISPR delivery methods") and the Nodes/Edges filed under it.

12 Open / deferred decisions

- Whether `rashomon:process` ships in v1 or waits for the first real terminal Facet to design against.

- Same question for `rashomon:browser`'s multi-context isolation: build single-context/single-view first.
- Scripting/REPL Facet: deferred until the friction is concrete.
- `warg` adoption: revisit once it's past its early stage.
- Whether `contains` should ever allow multiple parents: currently single-parent by design. Multi-parent would cost real complexity: an ambiguous canonical location/breadcrumb for outline Facets, messier move and delete semantics (reference-counting instead of a clean single reparent or delete), and the risk of accidentally-created cycles in what's meant to be a strict hierarchy. The likely resolution isn't relaxing `contains` itself but leaning on `related-to`: an Entity genuinely relevant to a second Thread gets a `related-to` edge to it, and outline Facets render that as a secondary "Related" section, most of the multi-homing benefit without sacrificing a single canonical home.